

Pramo Jewels

Backend Architecture

Document ID: PJ-BEARCH-001
Version: 1.0
Status: Draft

Field	Value
Project	Pramo Jewels Premium E-Commerce Platform
Purpose	Define backend architecture, services and implementation standards
Audience	Backend Developers, Solution Architects, DevOps, AI Coding Agents
Version	1.0

1. Backend Vision

Build a secure, modular, API-first backend using Laravel that supports high availability, maintainability, and future expansion while providing reliable services for the web application and future mobile apps.

2. Technology Stack

Framework: Laravel 12 (PHP)
Database: MySQL
Cache & Sessions: Redis
Queue: Redis Queue
Storage: S3-compatible Object Storage
API Style: REST (JSON)
Authentication: Laravel Sanctum with OTP support

3. Project Structure

app/
Http/
Models/
Services/
Repositories/
Policies/
Jobs/
Events/
Listeners/
Notifications/
Console/
database/
routes/
config/

4. Domain Modules

Authentication & Users
Products & Categories
Inventory
Wishlist & Cart
Orders
Payments
Shipping & Courier
Reviews
Coupons
Notifications
Reports & Analytics
Administration

5. Request Lifecycle

HTTP Request → Middleware → Validation → Controller → Service Layer → Repository → Database / External Services → JSON Response.

6. Business Logic

Keep controllers thin. Implement business rules in dedicated service classes. Use repositories to abstract data access and maintain separation of concerns.

7. API Design

Expose versioned REST endpoints with consistent response formats, pagination, filtering, standardized error payloads, and OpenAPI-compatible documentation.

8. Security

Role-Based Access Control
Input validation
CSRF protection where applicable
Rate limiting
Password hashing
Encrypted sensitive data
Audit logging

9. Asynchronous Processing

Use queues for:
Email
SMS/OTP
WhatsApp notifications
Order events
Image optimization
Inventory synchronization

10. Error Handling & Logging

Use centralized exception handling, structured logs, correlation IDs, and meaningful API error codes. Record critical events for auditing.

11. Scalability

Design stateless services, optimize queries, use caching, paginate large datasets, and support horizontal scaling behind a load balancer.

12. Testing & Quality

Use PHPUnit/Pest for unit and feature tests, API integration tests, static analysis, code reviews, and CI pipelines before deployment.